

In the Name of God



Sharif University of Technology
Department of Electrical Engineering

Course Project: Introduction to Machine Learning

Phase 2: OOD Detection and Generation with NADE

Instructor:

Dr. S. Amini

Authors:

Kiarash Hamidich (402111056)

Aida Ariafar (402101193)

Winter 1404

Contents

1	Theoretical Questions	2
1.1	Parameter Efficiency	2
1.1.1	1. Complexity of the Fully Expressive Tabular Method	2
1.1.2	2. Complexity of NADE Architecture	2
1.1.3	3. Scaling Comparison and the Curse of Dimensionality	3
1.2	The Computational “Trick”	3
1.2.1	1. Derivation of the Recursive Update Rule	3
1.2.2	2. Complexity Analysis (Big-O Notation)	4
1.3	Maximum Likelihood and Loss Function	4
1.3.1	1. Equivalence of Negative Log-Likelihood and BCE	4
1.3.2	2. Binarization of MNIST	5
1.4	Sampling vs. Scoring	5
1.4.1	1. Sampling Algorithm (Generation)	5
1.4.2	2. Computational Complexity: Sampling vs. Scoring	5
2	Practical Implementation & Experiments	6
2.1	Experimental Setup	6
2.2	Task 2: Training Dynamics	6
2.3	Task 3: OOD Detection	7
2.3.1	Histogram Analysis	7
2.3.2	Average Log-Likelihoods (ID vs. OOD)	7
2.3.3	Quantitative Metric (AUROC)	8
2.4	Task 4: Generation	8

1 Theoretical Questions

1.1 Parameter Efficiency

In this section, we compare how many parameters are needed by a fully expressive tabular model versus NADE when modeling high-dimensional binary data. The main takeaway is simple: tabular models blow up exponentially with dimension, while NADE stays manageable thanks to structure and weight sharing.

1.1.1 1. Complexity of the Fully Expressive Tabular Method

Solution Analysis: Consider a binary input vector $\mathbf{x} = [x_1, x_2, \dots, x_D]^\top$ where each $x_d \in \{0, 1\}$. The sample space $\mathcal{X}$ contains all binary configurations of length D , so:

$$|\mathcal{X}| = 2^D \quad (1)$$

A tabular (fully expressive) method assigns a separate probability to each possible configuration $\mathbf{x}^{(i)} \in \mathcal{X}$:

$$p(\mathbf{x} = \mathbf{x}^{(i)}) = \theta_i, \quad i = 1, \dots, 2^D \quad (2)$$

Because probabilities must sum to one,

$$\sum_{i=1}^{2^D} \theta_i = 1 \quad (3)$$

so there are only $2^D - 1$ free parameters:

$$\mathcal{N}_{\text{tabular}} = 2^D - 1 \quad (4)$$

Remark (Why “−1” matters): This is the classic degrees-of-freedom result: a categorical distribution over K outcomes has $K - 1$ free parameters due to normalization.

1.1.2 2. Complexity of NADE Architecture

Solution Analysis: NADE factors the joint distribution with the chain rule:

$$p(\mathbf{x}) = \prod_{d=1}^D p(x_d \mid \mathbf{x}_{<d}) \quad (5)$$

and uses shared neural-network parameters to model all conditionals.

Let H be the hidden size. Trainable parameters are:

- **Input Weight Matrix (W):** $W \in \mathbb{R}^{H \times D}$ Parameters: HD
- **Output Weight Matrix (V):** $V \in \mathbb{R}^{D \times H}$ Parameters: DH
- **Hidden Bias ($\mathbf{c}$):** $\mathbf{c} \in \mathbb{R}^H$ Parameters: H
- **Output Bias ($\mathbf{b}$):** $\mathbf{b} \in \mathbb{R}^D$ Parameters: D

Thus:

$$\begin{aligned} \mathcal{N}_{\text{NADE}} &= HD + DH + H + D \\ &= 2HD + H + D \end{aligned} \quad (6)$$

Remark (Parameter sharing intuition): Even though there are D different conditionals, NADE does not build D separate networks. Instead, it reuses W and V , which is the reason it scales so much better.

1.1.3 3. Scaling Comparison and the Curse of Dimensionality

Comparison Analysis:

1. Tabular Method (Exponential):

$$\mathcal{N}_{\text{tabular}} \in \Theta(2^D)$$

For MNIST, $D = 28 \times 28 = 784$:

$$\mathcal{N}_{\text{tabular}} = 2^{784} - 1 \approx 10^{236}$$

which is far beyond anything we can store or learn.

2. NADE (Polynomial):

$$\mathcal{N}_{\text{NADE}} = 2HD + H + D$$

If H is constant: $\mathcal{N}_{\text{NADE}} \in \Theta(D)$ (linear). If H scales like $H = \alpha D$: $\mathcal{N}_{\text{NADE}} \in \Theta(D^2)$ (quadratic).

Conclusion: NADE replaces an exponential parameter explosion with a structured model whose parameter count grows at most polynomially. This is exactly why it is practical for images.

1.2 The Computational “Trick”

A key reason NADE is efficient is that it avoids recomputing the same matrix products repeatedly. Instead, it keeps a running hidden pre-activation and updates it cheaply as we move pixel-by-pixel.

1.2.1 1. Derivation of the Recursive Update Rule

Solution Derivation: Let a_d be the hidden pre-activation used for predicting pixel x_d :

$$a_d = W_{\cdot, < d} x_{< d} + c \quad (7)$$

Expanding it as a sum of columns:

$$a_d = \sum_{k=1}^{d-1} W_{\cdot, k} x_k + c \quad (8)$$

Then:

$$a_{d+1} = \sum_{k=1}^d W_{\cdot, k} x_k + c = \left(\sum_{k=1}^{d-1} W_{\cdot, k} x_k + c \right) + W_{\cdot, d} x_d \quad (9)$$

So we get the recursion:

$$a_{d+1} = a_d + W_{\cdot, d} x_d \quad (10)$$

Interpretation: We can think of a_d as an “accumulator” that summarizes everything we have seen so far. When we move from pixel d to $d+1$, we only need to add one column of W (or add nothing if $x_d = 0$).

1.2.2 2. Complexity Analysis (Big-O Notation)

A. Naive Approach (No recursion): At step d , computing $a_d = W_{\cdot, <d} x_{<d} + c$ costs:

$$O(H(d-1))$$

Summed over all d :

$$C_{\text{naive}} = \sum_{d=1}^D O(Hd) = O\left(H \cdot \frac{D(D+1)}{2}\right) \in O(HD^2) \quad (11)$$

B. NADE with recursion: At each d , we:

- Update $a_{d+1} = a_d + W_{\cdot, d} x_d$: cost $O(H)$,
- Compute the conditional using a dot product with length H : cost $O(H)$.

So per dimension it is $O(H)$, and over D pixels:

$$C_{\text{NADE}} \in O(HD) \quad (12)$$

Conclusion: The recursion saves roughly a factor of D compared to the naive approach, which matters a lot when $D = 784$ for MNIST.

1.3 Maximum Likelihood and Loss Function

Here we connect the math to what we actually implement: training NADE by maximum likelihood is the same as minimizing a summed binary cross-entropy over pixels.

1.3.1 1. Equivalence of Negative Log-Likelihood and BCE

Using the chain rule:

$$p(\mathbf{x}) = \prod_{d=1}^D p(x_d \mid \mathbf{x}_{<d}) \quad (13)$$

For binary $x_d \in \{0, 1\}$, each conditional is Bernoulli with parameter $\hat{x}_d$:

$$p(x_d \mid \mathbf{x}_{<d}) = (\hat{x}_d)^{x_d} (1 - \hat{x}_d)^{1-x_d} \quad (14)$$

So:

$$\mathcal{L}_{\text{NLL}}(\mathbf{x}) = -\log p(\mathbf{x}) = -\sum_{d=1}^D \log p(x_d \mid \mathbf{x}_{<d}) \quad (15)$$

and:

$$\log p(x_d \mid \mathbf{x}_{<d}) = x_d \log \hat{x}_d + (1 - x_d) \log(1 - \hat{x}_d) \quad (16)$$

Thus:

$$\mathcal{L}_{\text{NLL}}(\mathbf{x}) = -\sum_{d=1}^D [x_d \log \hat{x}_d + (1 - x_d) \log(1 - \hat{x}_d)] \quad (17)$$

which is exactly the summed BCE loss.

Remark (Numerical stability): In practice, it is safer to compute BCE using logits (e.g., BCE-with-logits) to avoid numerical issues like $\log(0)$.

1.3.2 2. Binarization of MNIST

Why binarize MNIST?

- **Matches the likelihood:** NADE (in its basic form) assumes Bernoulli outputs, so it expects pixels in $\{0, 1\}$.
- **Keeps the story consistent:** MNIST is grayscale, but binarization makes the modeling assumption (binary pixels) and the data agree.

What would go wrong without binarization? If we keep pixels continuous in $[0, 1]$, a Bernoulli likelihood is no longer a correct observation model. You could still train something, but the probabilistic interpretation becomes mismatched, and likelihood values become less meaningful. A better option would be using a distribution designed for continuous data (or a discretized likelihood).

1.4 Sampling vs. Scoring

NADE can do two things: (1) *score* an existing image by computing its likelihood, and (2) *generate* a new image by sampling pixels one-by-one. The second is slower because it cannot be parallelized across pixels.

1.4.1 1. Sampling Algorithm (Generation)

Algorithm Description:

1. **Initialize:** start with $\mathbf{x} = \mathbf{0}$ and set $\mathbf{a}_1 = \mathbf{c}$.
2. **For** $d = 1$ **to** D :
 - Compute $\mathbf{h}_d = \sigma(\mathbf{a}_d)$.
 - Compute $\hat{x}_d = \sigma(\mathbf{V}_{d,:}^\top \mathbf{h}_d + b_d)$.
 - Sample $x_d \sim \text{Bernoulli}(\hat{x}_d)$.
 - Update $\mathbf{a}_{d+1} = \mathbf{a}_d + W_{:,d} x_d$ (if $d < D$).
3. **Return** $\mathbf{x}$.

1.4.2 2. Computational Complexity: Sampling vs. Scoring

Analysis: Scoring evaluates all pixels simultaneously using known inputs, whereas sampling requires sequential computation. Because x_d depends on the previously generated pixels $x_{<d}$, the sampling process cannot be parallelized across the D dimensions of a single image.

Conclusion: Scoring can be computed efficiently in batches (lots of images at once), and the operations are deterministic. Sampling, on the other hand, is sequential across D pixels because each new pixel depends on the previous ones, so it is inherently slower.

2 Practical Implementation & Experiments

In this section, we describe our NADE implementation and the experiments on MNIST for (1) OOD detection and (2) sample generation. The central idea is to use the model’s log-likelihood as an anomaly score: if an image looks “unfamiliar” to the model, its likelihood should drop.

2.1 Experimental Setup

- **Dataset:** MNIST handwritten digits (28×28 pixels, so $D = 784$).
- **Preprocessing:** Pixels are binarized with threshold 0.5 (values become 0 or 1).
- **In-Distribution vs. Out-of-Distribution:**
 - **ID digits:** $\{0, 1, 2, 3, 4\}$. The model is trained only on these classes.
 - **OOD digits:** $\{5, 6, 7, 8, 9\}$. These are never used for training and are only used for evaluation.
- **Train/Validation split:** From the ID data (0–4), we use an 80%/20% split for training and validation.
- **Model/Training hyperparameters (used in our experiments):**
 - Hidden Dimension (H): 500
 - Optimizer: Adam, learning rate 10^{-3}
 - Epochs: 10

2.2 Task 2: Training Dynamics

We train NADE by minimizing the Negative Log-Likelihood (NLL), which for binary data matches a summed BCE loss over pixels. Figure 1 shows the training and validation NLL over epochs.

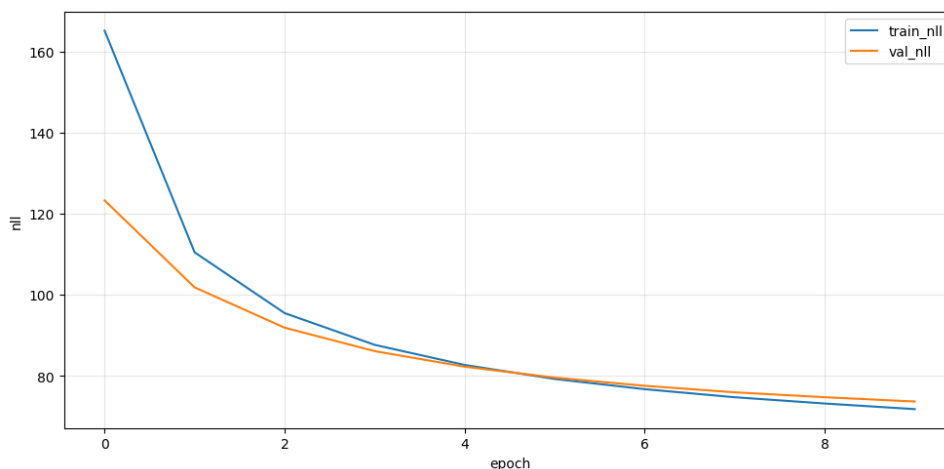


Figure 1: Training and Validation Negative Log-Likelihood (NLL) over epochs. Both curves decrease steadily, suggesting the model is learning useful structure from digits 0–4.

A nice sign here is that validation loss stays close to training loss. That usually means the model is learning general patterns (stroke shapes, digit structure) rather than simply memorizing the training images.

2.3 Task 3: OOD Detection

We use the model’s **log-likelihood (LL)** as an anomaly score. Since NADE is trained on digits 0–4, we expect:

- **Higher LL (lower NLL)** for ID samples.
- **Lower LL (higher NLL)** for OOD samples.

2.3.1 Histogram Analysis

Figure 2 shows overlaid histograms of log-likelihoods for ID and OOD.

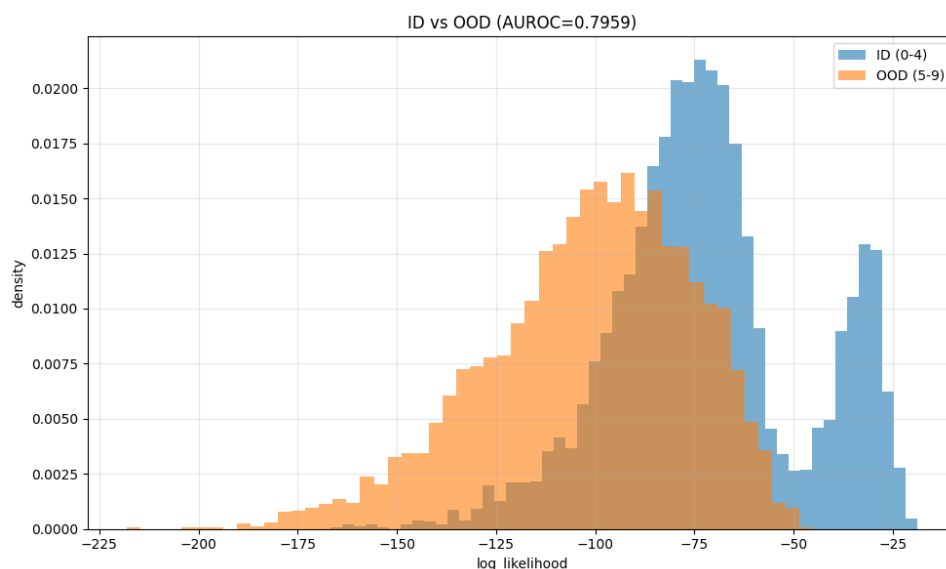


Figure 2: Histogram of log-likelihoods for ID (digits 0–4) vs. OOD (digits 5–9). ID samples are generally shifted to higher likelihood.

Observation: The ID distribution is shifted to the right (higher likelihood), while the OOD distribution is shifted to the left. This is exactly what we want: digits 5–9 tend to look “less familiar” to a model trained only on 0–4.

2.3.2 Average Log-Likelihoods (ID vs. OOD)

Besides the histogram shape, it helps to summarize the separation with a single number: the average LL. In our experiment, we obtain:

Split	Average Log-Likelihood
ID test (0–4)	−71.7008
OOD test (5–9)	−101.6542

The gap is quite clear: OOD samples receive a much lower (more negative) log-likelihood on average, which matches the histogram separation.

2.3.3 Quantitative Metric (AUROC)

To quantify how well likelihood separates the two groups, we report AUROC. We label:

- ID (0–4): 0
- OOD (5–9): 1

and we use NLL as the score (because OOD should have higher NLL).

Achieved AUROC: 0.7959

AUROC is 0.5 for random guessing and 1.0 for perfect separation. Our score shows that NADE likelihood is a strong signal for OOD detection, even though there is still some overlap between the two distributions (which is expected in a realistic setting).

2.4 Task 4: Generation

Finally, we generate new samples from the trained NADE model. Since the model only learned digits 0–4, we expect most generated samples to resemble those classes.

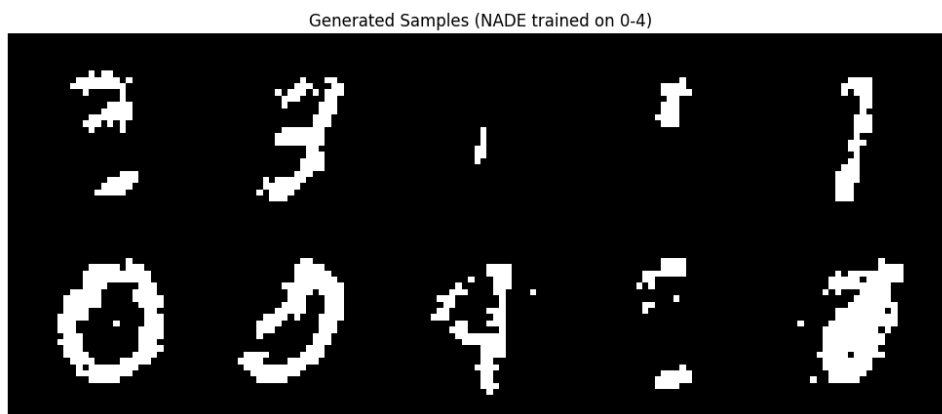


Figure 3: Samples generated by the NADE model. Most generated digits resemble the training classes (0–4).

Quality Discussion: Overall, the generated images capture recognizable digit-like structure.

- Because the model only sees digits 0–4 during training, it naturally tends to generate samples that look like those digits.
- The samples are binary, so a bit of speckle/noise is normal. NADE samples each pixel from a Bernoulli distribution, and small randomness at each step can accumulate across 784 pixel decisions.